



FACULTAD DE INFORMÁTICA

Servicios Telemáticos

Documento de Prácticas SSTT

Noelia Carolina Valarezo Loayza Grupo 2.2

ncarolina.valarezo@um.es

DNI: 23310348D

Fátima Zahra Daoudi Daoudi Grupo 2.2

fz.daoudi@um.es

DNI: 30297694R

Profesor: ESTEBAN BELMONTE RODRIGUEZ

Fecha de entrega: 16/06/2026

Índice

1	Introducción	2
2	Web-SSTT HTTP Server	3
2.1	Función para enviar datos a través del socket	3
2.2	Función para recibir datos a través del socket	3
2.3	Función para cerrar la conexión TCP	3
2.4	Función para procesar <code>cookie_counter</code>	3
2.5	Función principal encargada de procesar las peticiones HTTP	4
2.5.1	Mecanismo de persistencia	4
2.5.2	Comprobaciones	4
2.5.3	Formularios y comprobación del correo electrónico	5
2.5.4	Gestión de cookies	5
2.5.5	Gestión de errores HTTP	5
2.5.6	Envío de respuestas	6
3	Servicio DNS	8
3.1	Persistencia DNS	8
3.2	DNS primario	8
3.3	DNS secundario	10
3.4	Verificación del funcionamiento	10
4	Servicio SMTP/POP	12
4.1	Configuración del servidor	12
4.2	Configuración del cliente	12
5	Servicio Apache HTTPS	17
5.1	Configuración del servidor	17
5.2	Verificación del servicio	18
6	Servicio IPsec	22
6.1	Descripción	22
6.2	Certificados utilizados	22
6.3	Configuración del servidor	22
6.4	Configuración del cliente	23
6.5	Verificación del funcionamiento	23
7	Trazas de Wireshark	26
7.1	Intercambio DNS + acceso HTTP	26
7.2	Intercambio DNS + acceso HTTPS	27
7.3	Intercambios DNS, SMTP y POP	30
8	Problemas encontrados	33
9	Conclusión y valoración personal	33

1. Introducción

En el siguiente documento se desarrollará la elaboración de las prácticas de la asignatura de “Servicios Telemáticos”. La práctica consiste en la configuración y puesta en marcha de un conjunto de servicios vistos en la asignatura, sobre el escenario de red que se muestra en la Figura 1.

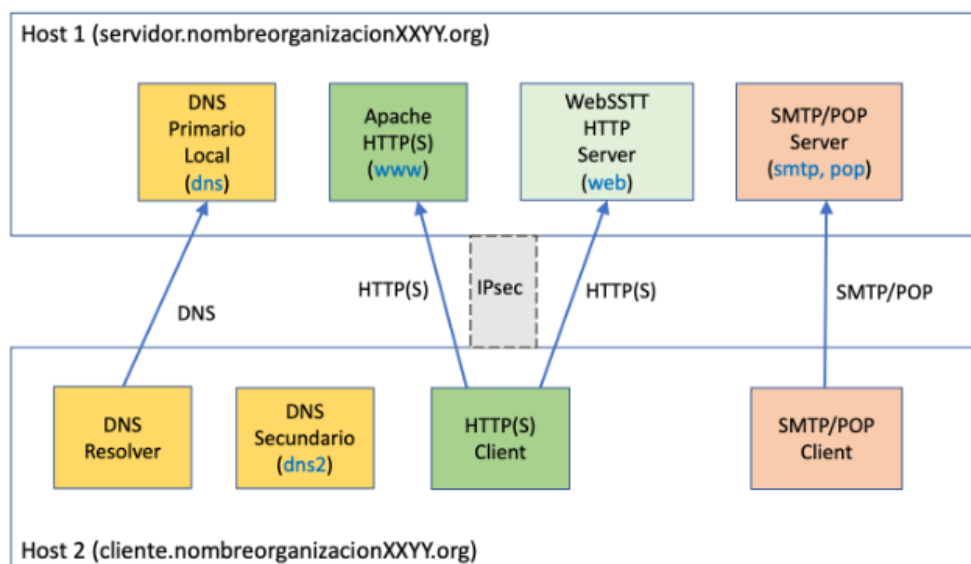


Figura 1: Escenario objetivo para SSTS.

La práctica se divide en:

- **Web-SSTS HTTP Server:** programación en Python de un servicio HTTP, con gestión de peticiones GET, formularios con el método GET, cookies, persistencia, verificación de peticiones y construcción de respuestas.
- **Servicio DNS:** configuración del servidor DNS local primario para la gestión de los distintos equipos desplegados, y un servidor DNS secundario que se actualiza periódicamente con la información del primario.
- **Servicio SMTP/POP:** configuración del servicio SMTP/POP en el servidor y un cliente para verificar la instalación.
- **Servicio Apache HTTP/HTTPS:** configuración del servidor Apache mediante autenticación y certificados X.509.
- **Servicio IPsec:** protección de la conexión cliente–servidor mediante una asociación de seguridad IPsec en modo transporte con IKEv2.

Para el correcto desarrollo de la práctica, se deberá tener en cuenta que:

El **host 1** (servidor) tendrá la dirección IP 192.168.56.101 y el **host 2** (cliente) tendrá la dirección IP 192.168.56.102.

2. Web-SSTT HTTP Server

El desarrollo del servidor se ha llevado a cabo mediante la plantilla facilitada por los recursos de la asignatura, que hace uso del módulo `socket` de Python, el cual proporciona una interfaz a la API de *sockets* de Berkeley. En la plantilla se han completado las siguientes funciones:

2.1. Función para enviar datos a través del socket

Recibe por parámetros el socket por el que enviar y el *buffer* con los datos a escribir. Los codifica en binario y los envía. Devuelve el número de bytes enviados.

```
def enviar_mensaje(cs, data):  
    return cs.send(data.encode())
```

2.2. Función para recibir datos a través del socket

Recibe por parámetros el socket por el que llegarán los datos. Realiza la lectura de la información en formato binario hasta el tamaño máximo determinado por el búfer y la transforma mediante decodificación a una cadena de caracteres. Devuelve una cadena con los datos.

```
def recibir_mensaje(cs):  
    return cs.recv(BUFSIZE).decode()
```

2.3. Función para cerrar la conexión TCP

```
def cerrar_conexion(cs):  
    cs.close()
```

2.4. Función para procesar cookie_counter

Analiza las cabeceras que recibe por parámetro para encontrar la cabecera `Cookie`. Si es la primera conexión y el cliente no incluye dicha cabecera, el servidor asigna el valor 1. En caso contrario, incrementa el valor de `cookie_counter` en una unidad hasta alcanzar el máximo de conexiones permitidas (`MAX_ACCESOS`).

```
def process_cookies(headers, cs):  
    for cabecera in headers:  
        nombre = cabecera.split(": ")[0]  
        if nombre == "Cookie":  
            cookie_counter = int(cabecera.split("=")[1])  
            if cookie_counter == MAX_ACCESOS:  
                return MAX_ACCESOS  
            else:  
                return cookie_counter + 1  
    return 1
```

2.5. Función principal encargada de procesar las peticiones HTTP

2.5.1. Mecanismo de persistencia

El servidor implementa conexiones persistentes mediante la cabecera `Connection: keep-alive`. La función permanece en ejecución mediante un bucle que permite atender varias peticiones del mismo cliente en una única conexión TCP. La persistencia se controla con `select.select()`, que espera durante un tiempo máximo (`TIMEOUT_CONNECTION`) la llegada de nuevas peticiones:

- Si se reciben datos, la petición se procesa y el bucle continúa.
- Si se agota el *timeout*, se considera la conexión inactiva, se cierra el socket y finaliza la función.
- Si `recv()` devuelve una cadena vacía (TCP FIN del cliente), también se cierra la conexión y finaliza la función.

```
while True:
    rlist = [cs]
    rsublist, _, _ = select.select(rlist, [], [],
        TIMEOUT_CONNECTION)
    if rsublist:
        data = recibir_mensaje(cs)
        if not data:
            cerrar_conexion(cs)
            logger.info("Cerrando conexion...\n")
            return None
    else:
        cerrar_conexion(cs)
        logger.info("Timeout\n")
        return None
```

2.5.2. Comprobaciones

1. Se comprueba que la línea de solicitud tenga el formato correcto (*método* – *URL* – *versión*). Si no se cumple, se responde con 400 Bad Request.
2. Se verifica que la versión sea HTTP/1.1; en caso contrario se devuelve 505 HTTP Version Not Supported.

```
if version == "HTTP/1.1":
    ...
else:
    codigo = "505 HTTP Version Not Supported"
```

3. Se comprueba que el método sea GET o POST. Cualquier otro método genera 405 Method Not Allowed.

```
if metodo in ['GET', 'POST']:
    ...
else:
    codigo = "405 Method Not Allowed"
```

4. Se procesa la URL y se construye la ruta absoluta del recurso solicitado combinando el directorio raíz (*webroot*) con el recurso de la URL.

5. Se verifica la presencia de la cabecera `Host`. Si no está presente, se devuelve `400 Bad Request`.
6. Se comprueba que el recurso solicitado existe en el sistema de ficheros. Si no existe, se devuelve `404 Not Found`.

```
if os.path.isfile(ruta):  
    ...  
else:  
    codigo = "404 Not Found"
```

2.5.3. Formularios y comprobación del correo electrónico

Cuando la petición contiene parámetros en la URL (formulario enviado con método GET), se extrae el valor del parámetro `email` y se comprueba si corresponde a uno de los correos válidos del grupo: `[fatima_94 | noelia_48]@nombreorganizacion9448.org`. Se devuelve una página HTML indicando si el correo es correcto o no.

```
if parametros:  
    for par in parametros.split("&"):  
        nombre, _, valor = par.partition("=")  
        if nombre == "email":  
            email = valor.replace("%40", "@")  
            if email in ["fatima_94@nombreorganizacion9448.org",  
                        "noelia_48@nombreorganizacion9448.org"]:  
                recurso = "/EmailCorrecto.html"  
            else:  
                recurso = "/EmailIncorrecto.html"
```

El símbolo `@` llega codificado como `%40` en la URL enviada por el cliente, por lo que es necesario decodificarlo antes de realizar la comparación.

2.5.4. Gestión de cookies

Para la gestión de cookies se utiliza la función `process_cookies()`, invocada únicamente cuando el recurso solicitado es `index.html`. Dicha función incrementa el valor de `cookie_counter` hasta alcanzar `MAX_ACCESOS`, en cuyo caso se devuelve `403 Forbidden`. El servidor indica al cliente que la cookie expira a los 30 segundos mediante la cabecera `Max-Age=30`.

```
if recurso == "/index.html":  
    cookie_counter = process_cookies(cabeceras, cs)  
    if cookie_counter == MAX_ACCESOS:  
        codigo = "403 Forbidden"  
else:  
    cookie_counter = None
```

2.5.5. Gestión de errores HTTP

Cuando se produce un error, el servidor intenta enviar una página HTML de error desde el directorio `webroot`. Si el fichero no existe, genera dinámicamente un cuerpo HTML con el

código de error. En todos los casos de error la respuesta incluye la cabecera `Connection: close`.

Los códigos de error gestionados son:

- **400 Bad Request:** mensaje de solicitud no entendido por el servidor.
- **403 Forbidden:** límite de accesos alcanzado.
- **404 Not Found:** documento solicitado no encontrado.
- **405 Method Not Allowed:** método HTTP no permitido.
- **505 HTTP Version Not Supported:** versión HTTP distinta de 1.1.

2.5.6. Envío de respuestas

Una vez construida la respuesta, se envían primero las cabeceras HTTP y después el contenido del fichero. El fichero se abre en modo binario y se envía en bloques de `BUFSIZE` bytes (8 KB) mediante un bucle, hasta que no queda contenido por enviar. Las respuestas HTTP se construyen siguiendo el orden indicado en el enunciado.

```
if codigo == "200":
    enviar_mensaje(cs, mensaje(codigo, tipo, tamano, fecha,
                              cookie_counter))
    with open(ruta, "rb") as f:
        while True:
            bloque = f.read(BUFSIZE)
            if not bloque:
                break
            cs.send(bloque)
else:
    codigo = codigo.split()[0]
    ruta_error = os.path.join(webroot, codigo + ".html")
    try:
        with open(ruta_error, "rb") as f:
            cuerpo = f.read()
    except FileNotFoundError:
        cuerpo = ("<html><body><h1>{}</h1></body></html>"
                  .format(codigo).encode())
    tamano = len(cuerpo)
    tipo = "text/html"
    enviar_mensaje(cs, mensaje(codigo, tipo, tamano, fecha, None)
                  )
    cs.send(cuerpo)
```

Las respuestas para `index.html` incluyen la cabecera `Set-Cookie`; el resto de recursos y las respuestas de error no la llevan. Las respuestas de error tampoco incluyen la cabecera `Keep-Alive`.

```
def mensaje(codigo, tipo, tamano, fecha, cookie):
    if codigo == "200":
        if cookie is not None:
            respuesta = "...".format(tipo, tamano, fecha,
                                     TIMEOUT_CONNECTION, cookie)
        else:
            respuesta = "...".format(tipo, tamano, fecha,
```

```
                                TIMEOUT_CONNECTION)
else:
    respuesta = "...".format(codigo, tipo, tamano, fecha)
return respuesta
```


3. Servicio DNS

A continuación se describe la configuración para desplegar los servicios DNS.

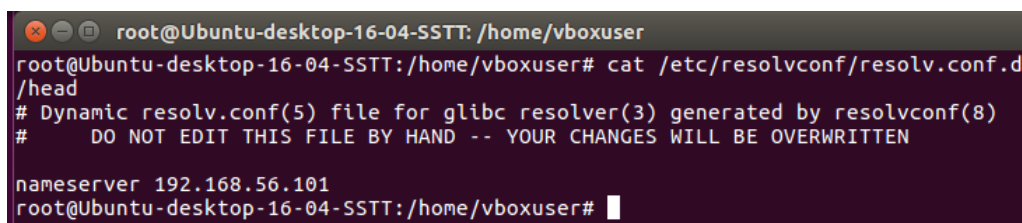
3.1. Persistencia DNS

Para añadir la IP del servidor DNS de forma persistente, se agrega la siguiente línea al final del fichero `/etc/resolvconf/resolv.conf.d/head` tanto en el servidor como en el cliente:

```
nameserver 192.168.56.101
```

A continuación se reinicia el servicio para que el cambio tenga efecto:

```
$ sudo systemctl restart resolvconf
```



```
root@Ubuntu-desktop-16-04-SSTT: /home/vboxuser
root@Ubuntu-desktop-16-04-SSTT:/home/vboxuser# cat /etc/resolvconf/resolv.conf.d/head
# Dynamic resolv.conf(5) file for glibc resolver(3) generated by resolvconf(8)
#     DO NOT EDIT THIS FILE BY HAND -- YOUR CHANGES WILL BE OVERWRITTEN

nameserver 192.168.56.101
root@Ubuntu-desktop-16-04-SSTT:/home/vboxuser#
```

Figura 2: Fichero `/etc/resolvconf/resolv.conf.d/head` con el `nameserver` configurado.

3.2. DNS primario

Para la configuración global de opciones se utiliza el archivo `/etc/bind/named.conf.options`. El directorio de trabajo es `/var/cache/bind`, donde se guarda la copia local de resoluciones aprendidas. La única modificación respecto al fichero original es la del parámetro `dnssec-validation`, que pasa de `auto` a `no`:

```
options {
    directory "/var/cache/bind";

    // forwarders {
    //     0.0.0.0;
    // };

    dnssec-validation no;

    auth-nxdomain no;      # conform to RFC1035
    listen-on-v6 { any; };
};
```

Listing 1: Fichero `named.conf.options` — DNS primario.

Para la configuración de zonas locales se edita `/etc/bind/named.conf.local`. Las opciones más relevantes son:

- **type master:** indica que este servidor es el DNS primario para la zona.
- **file:** nombre del archivo que contiene los datos de configuración de la zona.

- **allow-transfer:** lista de servidores esclavos autorizados a pedir una transferencia de zona.

```
zone "nombreorganizacion9448.org" {
    type master;
    file "/etc/bind/db.nombreorganizacion9448.org.zone";
    allow-transfer { 192.168.56.102; };
};
```

Listing 2: Fichero `named.conf.local` — DNS primario.

Para la configuración del fichero de zona se crea en `/etc/bind/` el archivo `db.nombreorganizacion9448.org.zone` tomando como referencia `/etc/bind/db.local`:

```
; BIND data file for local loopback interface
;
$TTL      604800
$ORIGIN   nombreorganizacion9448.org.
@   IN    SOA  servidor.nombreorganizacion9448.org. admin.
        nombreorganizacion9448.org. (
                2           ; Serial
        604800           ; Refresh
        86400            ; Retry
        2419200          ; Expire
        604800 )          ; Negative Cache TTL
;
@   IN    NS   servidor.nombreorganizacion9448.org.
@   IN    NS   cliente.nombreorganizacion9448.org.
@   IN    MX   10 smtp.nombreorganizacion9448.org.

servidor   IN  A   192.168.56.101
cliente    IN  A   192.168.56.102
dns1       IN  A   192.168.56.101
dns2       IN  A   192.168.56.102
www        IN  A   192.168.56.101
web        IN  A   192.168.56.101
smtp       IN  A   192.168.56.101
pop        IN  A   192.168.56.101
```

Listing 3: Fichero `db.nombreorganizacion9448.org.zone`.

A continuación se describe el significado de las directivas y registros más importantes:

- **\$TTL:** ajusta el tiempo (en segundos) durante el que un RR (*Resource Record*) de zona es válido. Se ha tomado el valor del fichero `db.local` de ejemplo.
- **\$ORIGIN:** anexa el nombre del dominio a los registros no cualificados.
- **Registro SOA:** indica el inicio de los datos para una zona y define parámetros que afectan a todos sus registros. El símbolo `@` es sustituido por el valor de `$ORIGIN`. Se ha indicado `servidor.nombreorganizacion9448.org` como servidor autoritativo y `admin.nombreorganizacion9448.org` como dirección del administrador.
- **Registro NS:** identifica a los servidores de nombres de la zona. Se han definido tanto el primario (*servidor*) como el secundario (*cliente*).
- **Registro MX:** identifica el servidor de correo del dominio, con prioridad 10.

- **Registro A:** convierte un nombre en una dirección IPv4. Se han resuelto todos los nombres indicados en el enunciado.

3.3. DNS secundario

Para el DNS secundario, desplegado en `cliente.nombreorganizacion9448.org`, se modifican los siguientes ficheros.

El fichero `/etc/bind/named.conf.options` es idéntico al del servidor primario (véase la sección anterior).

En el fichero `/etc/bind/named.conf.local` se declara la zona como esclava:

```
zone "nombreorganizacion9448.org." {  
    type slave;  
    file "/var/cache/bind/db.nombreorganizacion9448.org.zone";  
    masters { 192.168.56.101; };  
};
```

Listing 4: Fichero `named.conf.local` — DNS secundario.

Los campos más relevantes de esta configuración son:

- **type slave:** indica que este servidor es el DNS secundario para la zona.
- **file:** ruta donde se almacenará la copia de la zona recibida del primario.
- **masters:** dirección IP del servidor primario del que se sincroniza la zona.

3.4. Verificación del funcionamiento

Para verificar el funcionamiento se ejecutan los comandos `host` y `dig`.

El comando `host` realiza consultas simples. A continuación se muestra la salida para los distintos registros configurados:

```
root@Ubuntu-server-16-04-SSTT:/home/vboxuser# host -t ns nombreorganizacion9448.org  
nombreorganizacion9448.org name server servidor.nombreorganizacion9448.org.  
nombreorganizacion9448.org name server cliente.nombreorganizacion9448.org.  
root@Ubuntu-server-16-04-SSTT:/home/vboxuser# host -t mx nombreorganizacion9448.org  
nombreorganizacion9448.org mail is handled by 10 smtp.nombreorganizacion9448.org.  
root@Ubuntu-server-16-04-SSTT:/home/vboxuser# host servidor.nombreorganizacion9448.org  
servidor.nombreorganizacion9448.org has address 192.168.56.101  
root@Ubuntu-server-16-04-SSTT:/home/vboxuser# host cliente.nombreorganizacion9448.org  
cliente.nombreorganizacion9448.org has address 192.168.56.102  
root@Ubuntu-server-16-04-SSTT:/home/vboxuser# host dns1.nombreorganizacion9448.org  
dns1.nombreorganizacion9448.org has address 192.168.56.101  
root@Ubuntu-server-16-04-SSTT:/home/vboxuser# host dns2.nombreorganizacion9448.org  
dns2.nombreorganizacion9448.org has address 192.168.56.102  
root@Ubuntu-server-16-04-SSTT:/home/vboxuser# host www.nombreorganizacion9448.org  
www.nombreorganizacion9448.org has address 192.168.56.101  
root@Ubuntu-server-16-04-SSTT:/home/vboxuser# host web.nombreorganizacion9448.org  
web.nombreorganizacion9448.org has address 192.168.56.101  
root@Ubuntu-server-16-04-SSTT:/home/vboxuser# host smtp.nombreorganizacion9448.org  
smtp.nombreorganizacion9448.org has address 192.168.56.101  
root@Ubuntu-server-16-04-SSTT:/home/vboxuser# host pop.nombreorganizacion9448.org  
pop.nombreorganizacion9448.org has address 192.168.56.101
```

Figura 3: Salida del comando `host` para los registros de la zona.

Para obtener información más detallada se utiliza el comando `dig`:

```

root@Ubuntu-server-16-04-SST:/home/vboxuser# dig www.um.es

; <<>> DiG 9.10.3-P4-Ubuntu <<>> www.um.es
;; global options: +cmd
;; Got answer:
;; ->HEADER<<- opcode: QUERY, status: NOERROR, id: 50066
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 4, ADDITIONAL: 9

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;www.um.es.                IN      A

;; ANSWER SECTION:
www.um.es.                 28798   IN      CNAME   wwwclu.um.es.
wwwclu.um.es.             25830   IN      A       155.54.212.103

;; AUTHORITY SECTION:
um.es.                     83429   IN      NS      chico.rediris.es.
um.es.                     83429   IN      NS      dns1.um.es.
um.es.                     83429   IN      NS      dns2.um.es.
um.es.                     83429   IN      NS      sun.rediris.es.

;; ADDITIONAL SECTION:
sun.rediris.es.            83429   IN      A       199.184.182.1
sun.rediris.es.            83429   IN      AAAA    2620:171:808::1
dns1.um.es.                83428   IN      A       155.54.1.1
dns1.um.es.                83428   IN      AAAA    2001:720:1710:1::1
dns2.um.es.                83428   IN      A       155.54.1.2
dns2.um.es.                83428   IN      AAAA    2001:720:1710:1::2
chico.rediris.es.          83429   IN      A       162.219.54.2
chico.rediris.es.          83429   IN      AAAA    2620:10a:80eb::2

;; Query time: 0 msec
;; SERVER: 192.168.56.101#53(192.168.56.101)
;; WHEN: Thu Jun 11 23:13:38 CEST 2026
;; MSG SIZE rcvd: 335

```

Figura 4: Salida del comando `dig www.um.es`.

En la línea `SERVER` de la salida de `dig` se puede comprobar que la consulta ha sido resuelta por el DNS primario (192.168.56.101).

4. Servicio SMTP/POP

4.1. Configuración del servidor

Para configurar el servicio SMTP se instala `exim4` y se configura con los parámetros indicados a continuación:

- Tipo general del servidor: *Internet site*.
- Nombre del sistema de correo: `nombreorganizacion9448.org`.
- Direcciones IP en las que recibir conexiones SMTP (en blanco): cualquier IP.
- Destinos de los que se acepta correo: `nombreorganizacion9448.org`.
- Dominio para reenvío de correo: `nombreorganizacion9448.org`.
- Máquinas para las cuales reenviar correo: `192.168.56.0/24`.
- Limitar consultas DNS: NO.
- Formato de buzón de correo: *Maildir*.
- Dividir ficheros de configuración: NO.

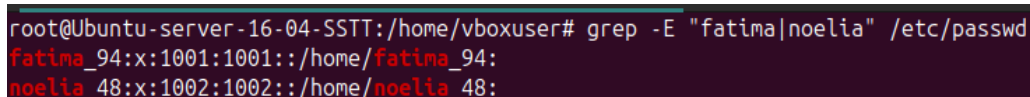
Tras la configuración, se reinicia el servicio SMTP:

```
# service exim4 restart
```

A continuación, se instala `dovecot-pop3d` para el servidor POP y se editan los siguientes ficheros:

- `/etc/dovecot/conf.d/10-auth.conf`: se establecen `disable_plaintext_auth = no` (para permitir autenticación en texto plano) y `auth_mechanisms = plain` (para activar el mecanismo de autenticación simple).
- `/etc/dovecot/conf.d/10-mail.conf`: se establece `mail_location = maildir:~/Maildir` para especificar el formato de los buzones de correo.

A continuación, se crean los usuarios del sistema con el flag `-m`, que indica que el buzón debe tener formato *Maildir*:



```
root@Ubuntu-server-16-04-SSTT:/home/vboxuser# grep -E "fatima|noelia" /etc/passwd
fatima_94:x:1001:1001::/home/fatima_94:
noelia_48:x:1002:1002::/home/noelia_48:
```

Figura 5: Usuarios `fatima_94` y `noelia_48` creados en el servidor.

Por último, se reinicia `dovecot`:

```
# service dovecot restart
```

4.2. Configuración del cliente

En el cliente no se modifica ningún archivo del sistema; únicamente se configura Thunderbird como cliente de correo. Se añaden las cuentas de los dos usuarios:

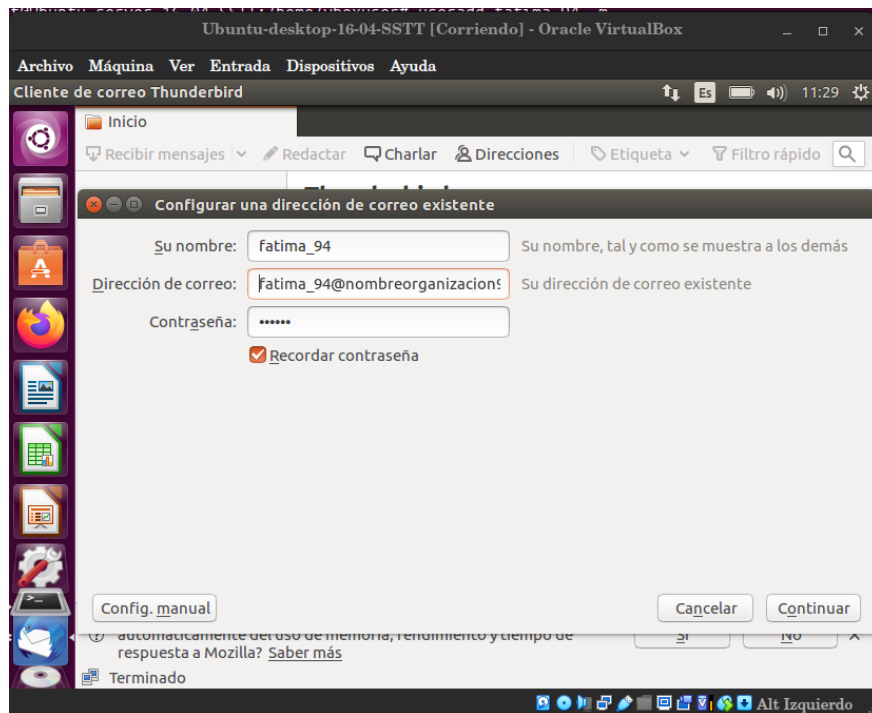


Figura 6: Creación de la cuenta del primer usuario (fatima_94@nombreorganizacion9448.org).

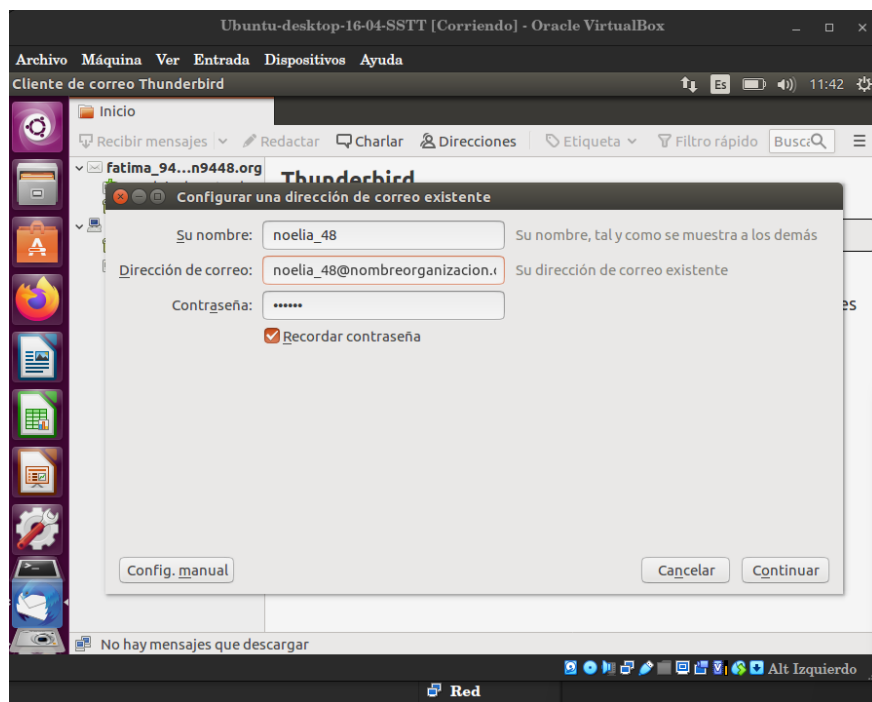


Figura 7: Creación de la cuenta del segundo usuario (noelia_48@nombreorganizacion9448.org).

La pantalla principal de Thunderbird muestra ambas cuentas correctamente configuradas:

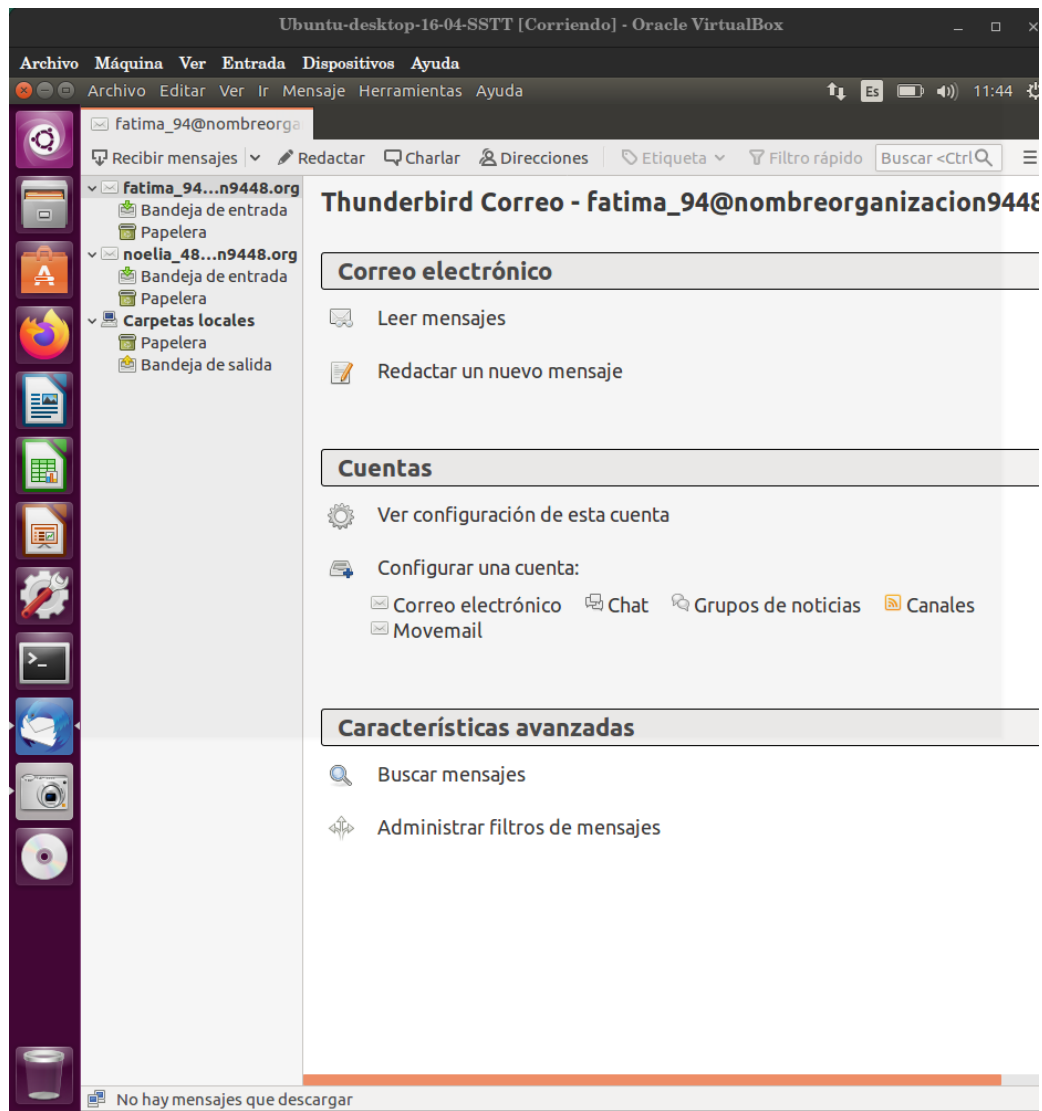


Figura 8: Pantalla principal de Thunderbird con las dos cuentas.

Para verificar el funcionamiento, se envía un correo desde el primer usuario al segundo:

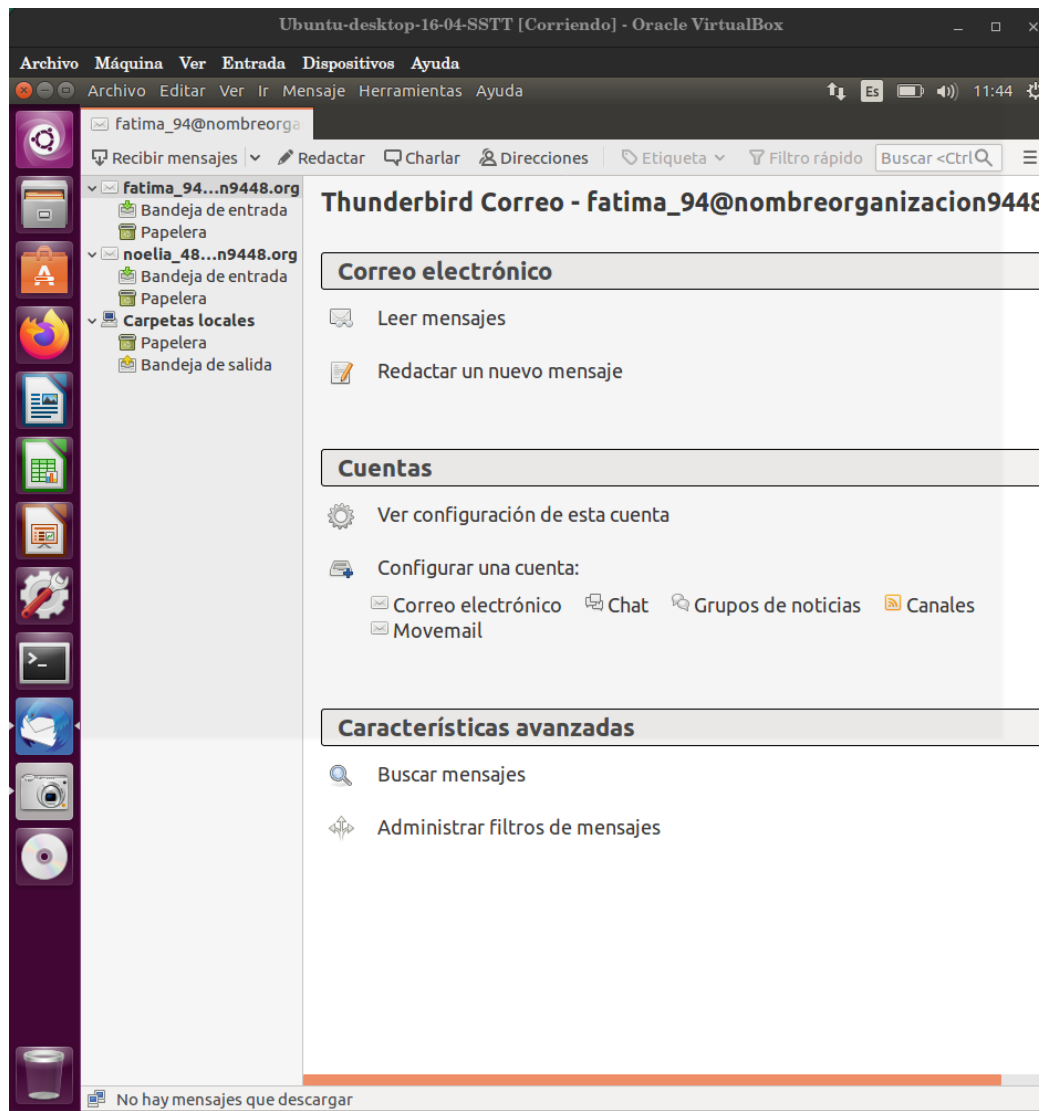


Figura 9: Bandeja de enviados del primer usuario (fatima_94).

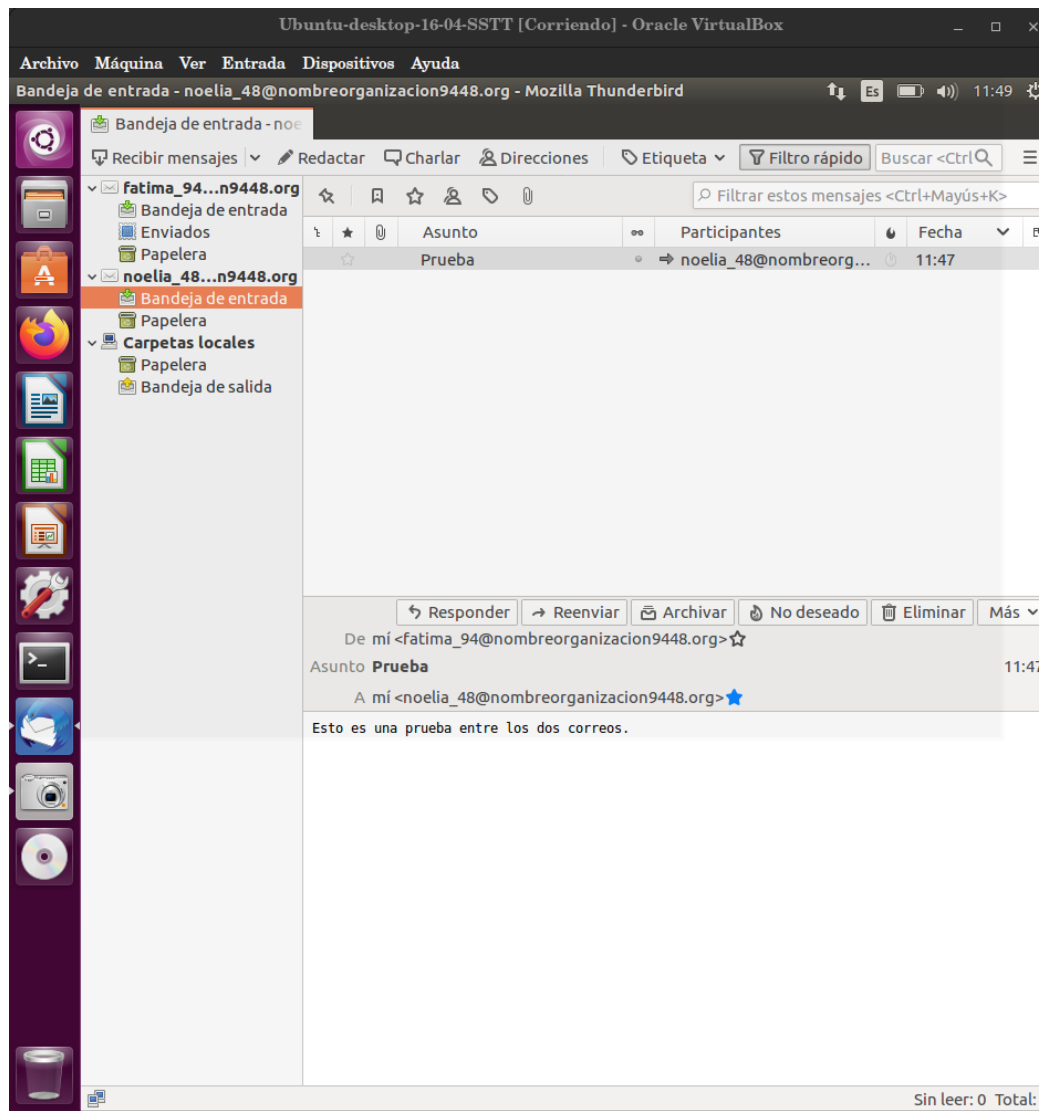


Figura 10: Bandeja de entrada del segundo usuario (noelia_48).

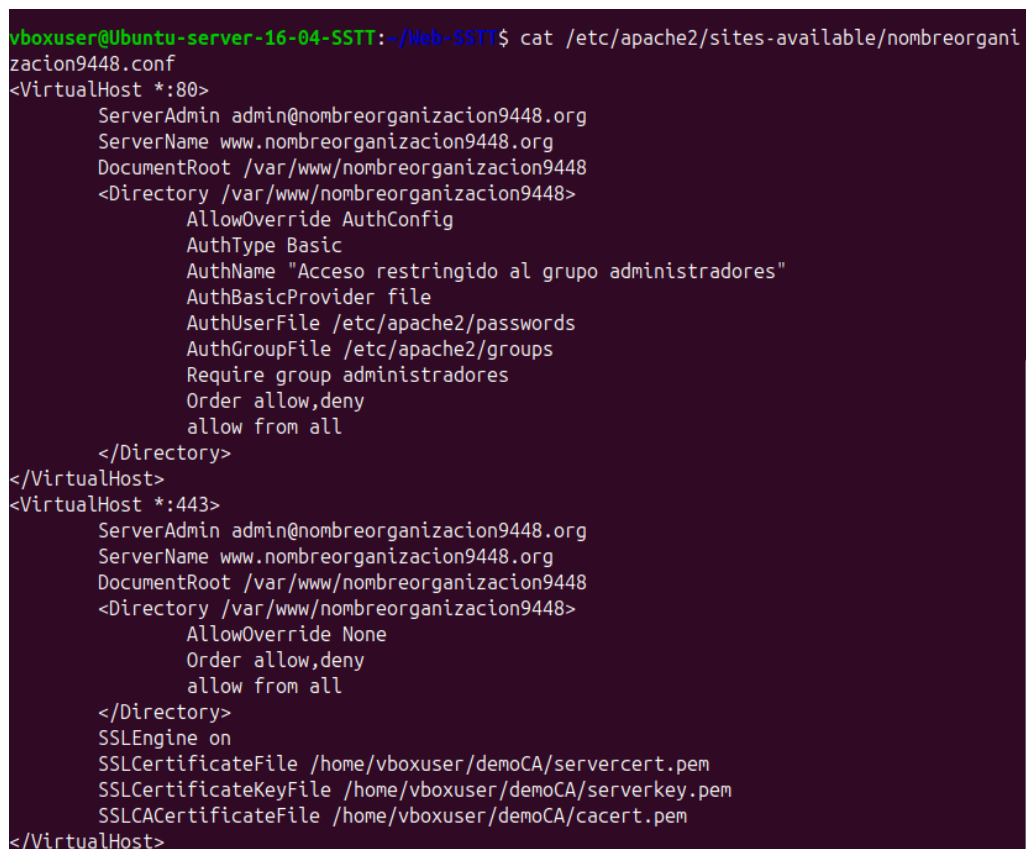
5. Servicio Apache HTTPS

5.1. Configuración del servidor

El servidor web Apache es el más utilizado en sistemas Linux, empleado para servir páginas web a clientes web. En primer lugar, se instala `apache2`:

```
# apt install apache2
```

A continuación, se crea el fichero de configuración de Virtual Hosts en `/etc/apache2/sites-available/` con el nombre `nombreorganizacion9448.conf`, para servir la web tanto por HTTP como por HTTPS:



```
vboxuser@Ubuntu-server-16-04-SSTT:~/Web-SSTT$ cat /etc/apache2/sites-available/nombreorgani
zacion9448.conf
<VirtualHost *:80>
    ServerAdmin admin@nombreorganizacion9448.org
    ServerName www.nombreorganizacion9448.org
    DocumentRoot /var/www/nombreorganizacion9448
    <Directory /var/www/nombreorganizacion9448>
        AllowOverride AuthConfig
        AuthType Basic
        AuthName "Acceso restringido al grupo administradores"
        AuthBasicProvider file
        AuthUserFile /etc/apache2/passwords
        AuthGroupFile /etc/apache2/groups
        Require group administradores
        Order allow,deny
        allow from all
    </Directory>
</VirtualHost>
<VirtualHost *:443>
    ServerAdmin admin@nombreorganizacion9448.org
    ServerName www.nombreorganizacion9448.org
    DocumentRoot /var/www/nombreorganizacion9448
    <Directory /var/www/nombreorganizacion9448>
        AllowOverride None
        Order allow,deny
        allow from all
    </Directory>
    SSLEngine on
    SSLCertificateFile /home/vboxuser/demoCA/servercert.pem
    SSLCertificateKeyFile /home/vboxuser/demoCA/serverkey.pem
    SSLCACertificateFile /home/vboxuser/demoCA/cacert.pem
</VirtualHost>
```

Figura 11: Configuración de Virtual Hosts en `nombreorganizacion9448.conf`.

Se puede observar que se ha indicado el directorio donde se guardarán las páginas HTML, `/var/www/nombreorganizacion9448`, reutilizando el fichero `index.html` y su imagen `logo-um.jpg` de prácticas anteriores. También se han añadido los ficheros de autenticación de grupos.

A continuación, se prepara el entorno para la PKI creando la carpeta de trabajo CA:

```
vboxuser@Ubuntu-server-16-04-SST:~/demoCA$ tree
.
|-- cacert.pem
|-- certs
|-- crinumber
|-- index.txt
|-- index.txt.attr
|-- index.txt.old
|-- newcerts
|   |-- 01.pem
|-- private
|   |-- cakey.pem
|-- serial
|-- serial.old
|-- servercert.pem
|-- servercsr.pem
|-- serverkey.pem
3 directories, 12 files
```

Figura 12: Estructura de la carpeta de trabajo CA.

Entre los ficheros generados cabe destacar los siguientes:

- **cakey.pem**: clave privada de la CA, protegida con contraseña.
- **cacert.pem**: certificado X.509 de la CA, que incluye su clave pública.
- **serverkey.pem**: clave privada RSA del servidor web.
- **servercsr.pem**: solicitud de certificación en formato PKCS#10.
- **serial**: fichero de número de serie, inicializado al valor 1.

5.2. Verificación del servicio

Para comprobar que todo funciona correctamente, se accede al servidor mediante HTTP introduciendo la siguiente URL en el navegador:

```
http://www.nombreorganizacion9448.org
```

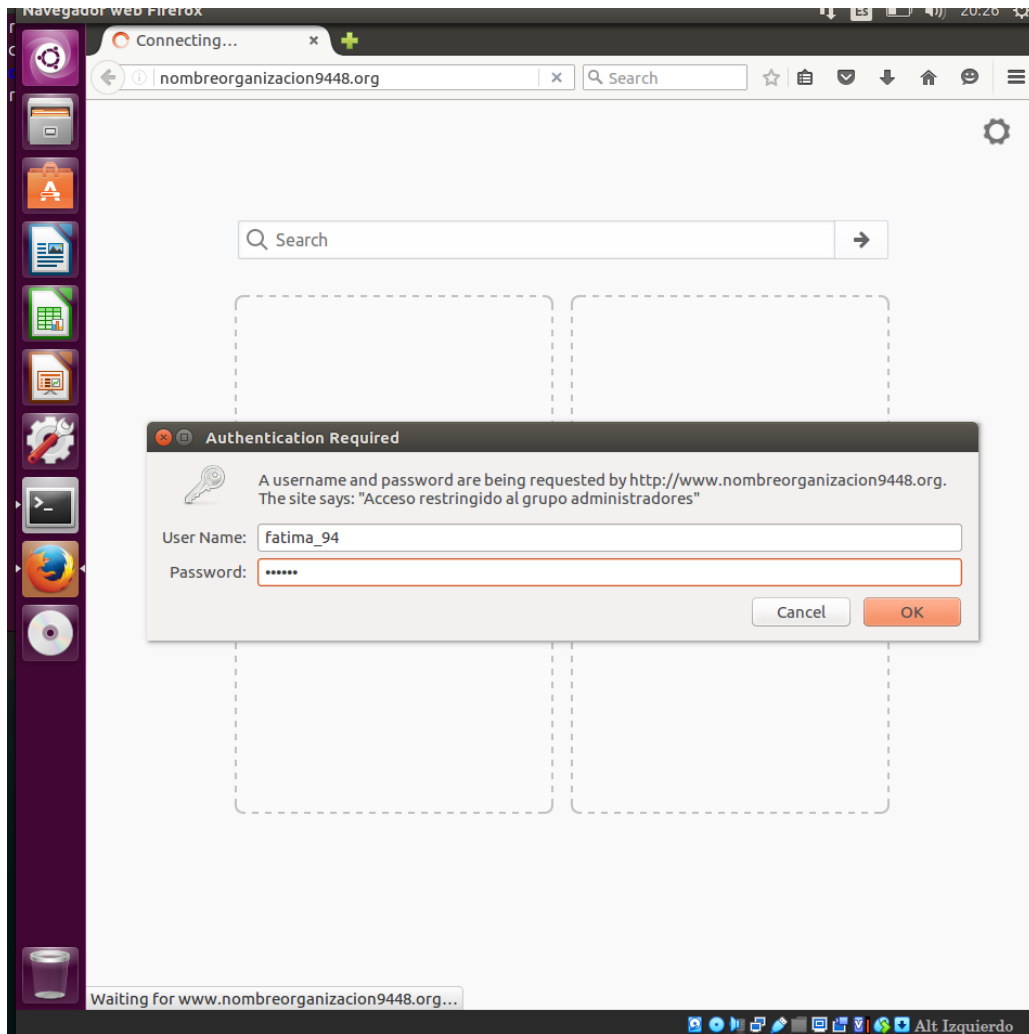


Figura 13: Acceso con petición de autenticación

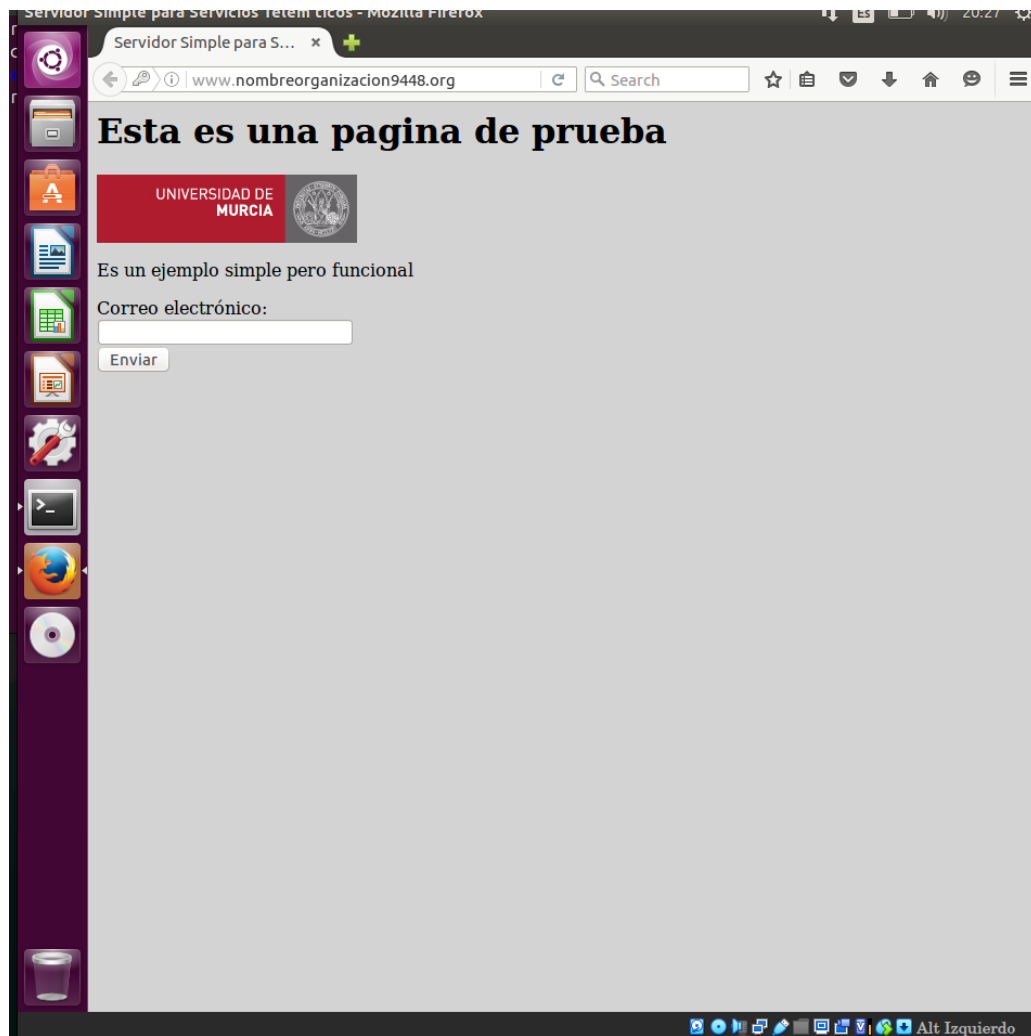


Figura 14: Acceso a la página web mediante HTTP.

Y para verificar el acceso seguro mediante HTTPS:

```
https://www.nombreorganizacion9448.org
```

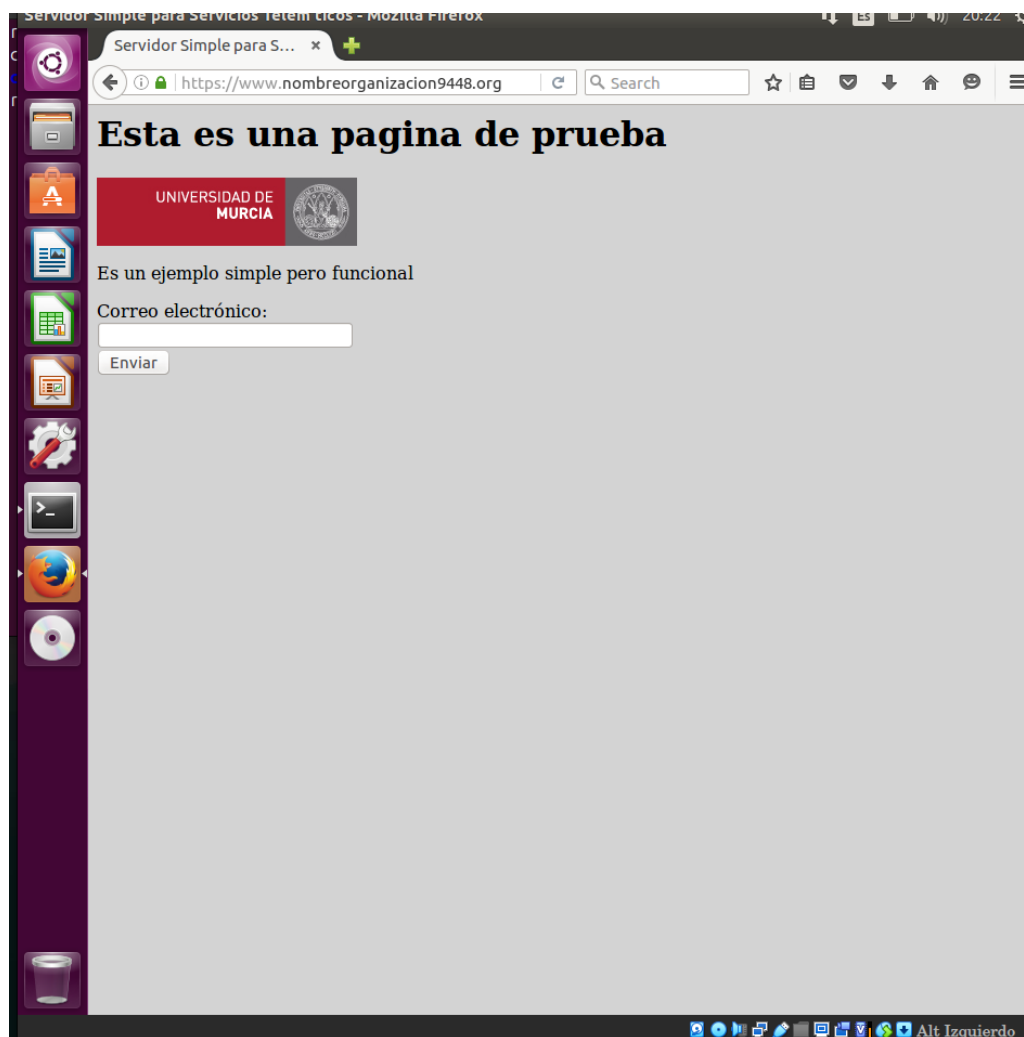


Figura 15: Acceso a la página web mediante HTTPS.

6. Servicio IPsec

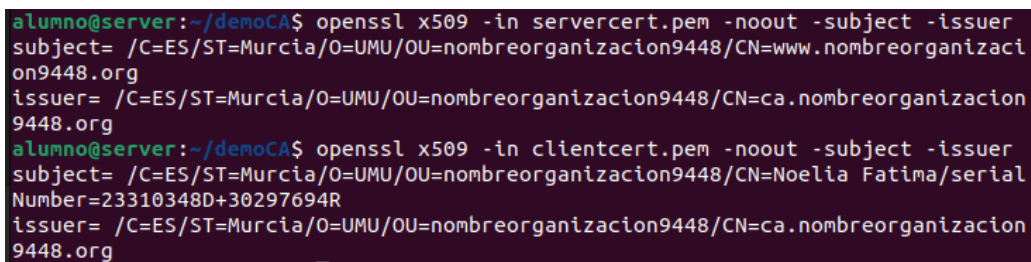
6.1. Descripción

Para proteger la comunicación entre el servidor (192.168.56.101) y el cliente (192.168.56.102) se ha configurado una asociación de seguridad IPsec en **modo transporte** utilizando **IKEv2** para la negociación automática de claves. La autenticación de ambos extremos se realiza mediante **certificados X.509**. Para la protección del tráfico se utiliza la cabecera **AH** (*Authentication Header*), que proporciona integridad y autenticación de origen sin cifrado. El software utilizado es **strongswan** (versión 5.3.5), instalado en ambas máquinas.

6.2. Certificados utilizados

Se han generado tres certificados firmados por una CA propia:

- **Certificado de la CA:** CN=ca.nombreorganizacion9448.org
- **Certificado del servidor:** CN=www.nombreorganizacion9448.org
- **Certificado del cliente.** CN=Noelia Fatima, SN=23310348D+30297694R



```
alumno@server:~/demoCA$ openssl x509 -in servercert.pem -noout -subject -issuer
subject= /C=ES/ST=Murcia/O=UMU/OU=nombreorganizacion9448/CN=www.nombreorganizaci
on9448.org
issuer= /C=ES/ST=Murcia/O=UMU/OU=nombreorganizacion9448/CN=ca.nombreorganizacion
9448.org
alumno@server:~/demoCA$ openssl x509 -in clientcert.pem -noout -subject -issuer
subject= /C=ES/ST=Murcia/O=UMU/OU=nombreorganizacion9448/CN=Noelia Fatima/serial
Number=23310348D+30297694R
issuer= /C=ES/ST=Murcia/O=UMU/OU=nombreorganizacion9448/CN=ca.nombreorganizacion
9448.org
```

Figura 16: Verificación de los certificados del servidor y del cliente.

6.3. Configuración del servidor

Los certificados se colocan en los directorios de Strongswan:

- /etc/ipsec.d/cacerts/ — certificado de la CA
- /etc/ipsec.d/certs/ — certificados del servidor y cliente
- /etc/ipsec.d/private/ — clave privada del servidor

```

alumno@server:~/demoCA$ cat /etc/ipsec.conf
# ipsec.conf - strongSwan IPsec configuration file

# basic configuration

config setup

conn %default
    ikelifetime=60m
    keylife=20m
    rekeymargin=3m
    keyingtries=1
    mobike=no
    keyexchange=ikev2
    authby=pubkey

conn host-host
    left=192.168.56.101
    leftcert=/etc/ipsec.d/certs/servercert.pem
    leftid="C=ES, ST=Murcia, O=UMU, OU=nombrequizacion9448, CN=www.nombrequizacion9448.org"
    right=192.168.56.102
    rightid="C=ES, ST=Murcia, O=UMU, OU=nombrequizacion9448, CN=Noelia Fatima, SN=23310348D+30297694R"
    type=transport
    ah=sha256
    auto=start
alumno@server:~/demoCA$ sudo cat /etc/ipsec.secrets
[sudo] password for alumno:
# This file holds shared secrets or RSA private keys for authentication.

# RSA private key for this host, authenticating it to any other host
# which knows the public part.

: RSA /etc/ipsec.d/private/serverkey.pem

```

Figura 17: Fichero /etc/ipsec.conf del servidor.

6.4. Configuración del cliente

```

alumno@desktop:~$ cat /etc/ipsec.conf
# ipsec.conf - strongSwan IPsec configuration file

# basic configuration

config setup

conn %default
    ikelifetime=60m
    keylife=20m
    rekeymargin=3m
    keyingtries=1
    mobike=no
    keyexchange=ikev2
    authby=pubkey

conn host-host
    left=192.168.56.102
    leftcert=/etc/ipsec.d/certs/clientcert.pem
    leftid="C=ES, ST=Murcia, O=UMU, OU=nombrequizacion9448, CN=Noelia Fatima, SN=23310348D+30297694R"
    right=192.168.56.101
    rightid="C=ES, ST=Murcia, O=UMU, OU=nombrequizacion9448, CN=www.nombrequizacion9448.org"
    type=transport
    ah=sha256
    auto=start
alumno@desktop:~$ sudo cat /etc/ipsec.secrets
[sudo] password for alumno:
# This file holds shared secrets or RSA private keys for authentication.

# RSA private key for this host, authenticating it to any other host
# which knows the public part.

: RSA /etc/ipsec.d/private/clientkey.pem

```

Figura 18: Fichero /etc/ipsec.conf del cliente.

6.5. Verificación del funcionamiento

Una vez arrancado Strongswan en ambas máquinas con `sudo ipsec restart`, se verifica que la asociación de seguridad se ha establecido correctamente:


```

alumno@server:~/demoCA$ sudo nano /etc/ipsec.conf
alumno@server:~/demoCA$ sudo ipsec restart
Stopping strongSwan IPsec...
Starting strongSwan 5.3.5 IPsec [starter]...
alumno@server:~/demoCA$ sudo ipsec status
Security Associations (1 up, 0 connecting):
  host-host[2]: ESTABLISHED 8 seconds ago, 192.168.56.101[C=ES, ST=Murcia, O=UM
U, OU=nombrequizacion9448, CN=www.nombrequizacion9448.org]...192.168.56.10
2[C=ES, ST=Murcia, O=UMU, OU=nombrequizacion9448, CN=Noelia Fatima, SN=233103
48D+30297694R]
  host-host[2]: INSTALLED, TRANSPORT, reqid 2, AH SPIs: c0eea89d_i cfa43ecf_o
  host-host[2]: 192.168.56.101/32 === 192.168.56.102/32
alumno@server:~/demoCA$

```

Figura 19: Salida de `ipsec status` en servidor, confirmando la SA establecida.

```

alumno@desktop:~$ sudo nano /etc/ipsec.conf
alumno@desktop:~$ sudo ipsec restart
Stopping strongSwan IPsec...
Starting strongSwan 5.3.5 IPsec [starter]...
alumno@desktop:~$ sudo ipsec status
Security Associations (1 up, 0 connecting):
  host-host[1]: ESTABLISHED 17 seconds ago, 192.168.56.102[C=ES, ST=Murcia, O=U
MU, OU=nombrequizacion9448, CN=Noelia Fatima, SN=23310348D+30297694R]...192.1
68.56.101[C=ES, ST=Murcia, O=UMU, OU=nombrequizacion9448, CN=www.nombrequi
zacion9448.org]
  host-host[1]: INSTALLED, TRANSPORT, reqid 1, AH SPIs: cfa43ecf_i c0eea89d_o
  host-host[1]: 192.168.56.102/32 === 192.168.56.101/32
alumno@desktop:~$

```

Figura 20: Salida de `ipsec status` en cliente, confirmando la SA establecida.

La SA aparece como ESTABLISHED en modo TRANSPORT con cabecera AH, confirmando el correcto funcionamiento.

No.	Time	Source	Destination	Protocol	Length	Info
69	19.347449	192.168.56.102	192.168.56.101	ISAKMP	523	IKE_SA_INIT MID=00 Responder Response
71	19.351886	192.168.56.101	192.168.56.102	ISAKMP	238	IKE_AUTH MID=01 Initiator Request
73	19.356226	192.168.56.102	192.168.56.101	ISAKMP	60	IKE_AUTH MID=01 Responder Response
76	25.744266	192.168.56.102	192.168.56.101	ICMP	130	Echo (ping) request id=0x2d68, seq=1/256 (reply in 77)
77	25.744314	192.168.56.101	192.168.56.102	ICMP	130	Echo (ping) reply id=0x2d68, seq=1/256 (request in 76)
78	26.743808	192.168.56.102	192.168.56.101	ICMP	130	Echo (ping) request id=0x2d68, seq=2/512 (reply in 79)
79	26.743852	192.168.56.101	192.168.56.102	ICMP	130	Echo (ping) reply id=0x2d68, seq=2/512 (request in 78)
80	27.743928	192.168.56.102	192.168.56.101	ICMP	130	Echo (ping) request id=0x2d68, seq=3/768 (reply in 81)
81	27.743992	192.168.56.101	192.168.56.102	ICMP	130	Echo (ping) reply id=0x2d68, seq=3/768 (request in 80)
82	28.743773	192.168.56.102	192.168.56.101	ICMP	130	Echo (ping) request id=0x2d68, seq=4/1024 (reply in 83)
83	28.743831	192.168.56.101	192.168.56.102	ICMP	130	Echo (ping) reply id=0x2d68, seq=4/1024 (request in 82)
84	29.743719	192.168.56.102	192.168.56.101	ICMP	130	Echo (ping) request id=0x2d68, seq=5/1280 (reply in 85)
85	29.743763	192.168.56.101	192.168.56.102	ICMP	130	Echo (ping) reply id=0x2d68, seq=5/1280 (request in 84)

Frame 78: 130 bytes on wire (1040 bits), 130 bytes captured (1040 bits)

Ethernet II, Src: PcsCompu_c2:7a:23 (08:00:27:c2:7a:23), Dst: PcsCompu_5c:12:7a (08:00:27:5c:12:7a)

Internet Protocol Version 4, Src: 192.168.56.102, Dst: 192.168.56.101

Authentication Header

Next header: ICMP (1)

Length: 6 (32 bytes)

Reserved: 0000

AH SPI: 0xc0ccbf

AH Sequence: 2

AH ICV: 09b2d012aa012dc35290a547f03081a1c0a83865

Internet Control Message Protocol

Figura 21: Captura Wireshark mostrando el intercambio IKEv2 (ISAKMP) y el tráfico protegido mediante AH.

En la captura se distinguen claramente dos fases:

- **Paquetes ISAKMP (68–73):** negociación IKEv2 mediante los mensajes `IKE_SA_INIT` e `IKE_AUTH`, donde los dos extremos se autentican con sus certificados X.509 y acuerdan los parámetros criptográficos.
- **Paquetes ICMP (76–85):** tráfico de ping entre los dos hosts, protegido por la cabecera AH que garantiza la integridad y autenticación de cada paquete.

7. Trazas de Wireshark

7.1. Intercambio DNS + acceso HTTP

El cliente realiza la resolución DNS de `www.nombreorganizacion9448.org` antes de establecer la conexión HTTP. En la siguiente captura se puede observar el intercambio de mensajes DNS con filtro `dns || http`:

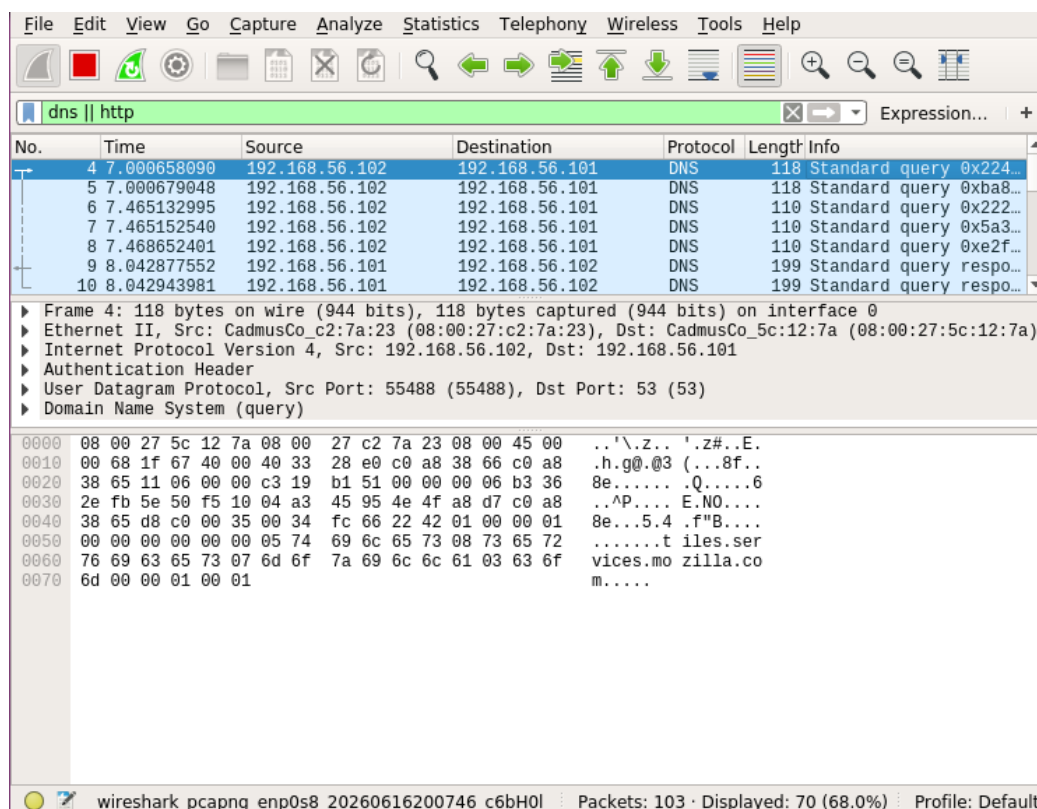


Figura 22: DNS Query para la resolución de `www.nombreorganizacion9448.org`.

Una vez resuelta la dirección, el cliente accede al servidor mediante HTTP. Se puede observar la petición `GET` al recurso `/logo-um.jpg` con respuesta `200 OK`, así como otros códigos de respuesta como `401 Unauthorized` al intentar acceder a recursos protegidos:

No.	Time	Source	Destination	Protocol	Length	Info
81	64.789581872	192.168.56.101	192.168.56.102	HTTP	872	HTTP/1.1 401 Unautho...
89	72.559876105	192.168.56.102	192.168.56.101	HTTP	536	GET / HTTP/1.1
91	72.560860004	192.168.56.101	192.168.56.102	HTTP	712	HTTP/1.1 200 OK (te...
93	72.671471079	192.168.56.102	192.168.56.101	HTTP	532	GET /logo-um.jpg HTT...
94	72.672386274	192.168.56.101	192.168.56.102	HTTP	279	HTTP/1.1 304 Not Mod...
96	72.696544975	192.168.56.102	192.168.56.101	HTTP	396	GET /favicon.ico HTT...
97	72.697586167	192.168.56.101	192.168.56.102	HTTP	606	HTTP/1.1 404 Not Fou...

Frame 93: 532 bytes on wire (4256 bits), 532 bytes captured (4256 bits) on interface 0

Ethernet II, Src: CadmusCo_c2:7a:23 (08:00:27:c2:7a:23), Dst: CadmusCo_5c:12:7a (08:00:27:5c:12:7a)

Internet Protocol Version 4, Src: 192.168.56.102, Dst: 192.168.56.101

Authentication Header

Transmission Control Protocol, Src Port: 59384 (59384), Dst Port: 80 (80), Seq: 439, Ack: 615, Len...

Hypertext Transfer Protocol

```

0000 08 00 27 5c 12 7a 08 00 27 c2 7a 23 08 00 45 00  ..'\.z...'.z#..E.
0010 02 06 f2 07 40 00 40 33 54 a1 c0 a8 38 66 c0 a8  ....@.3 T...8f..
0020 38 65 06 06 00 00 c3 19 b1 51 00 00 00 30 cf 9d  8e.....Q...0..
0030 b8 6d 18 5b c0 8e 61 e7 09 a4 92 38 92 bf c0 a8  .m.[..a. ...8...
0040 38 65 e7 f8 00 50 6c c9 cc a5 ce 47 b9 83 80 18  8e...Pl. ...G...
0050 00 ee 6e 58 00 00 01 01 08 0a 00 14 39 09 00 5b  ..nX.... ...9..[
0060 86 79 47 45 54 20 2f 6c 6f 67 6f 2d 75 6d 2e 6a  .yGET /l ogo-um.]
0070 70 67 20 48 54 54 50 2f 31 2e 31 0d 0a 48 6f 73  pg HTTP/ 1.1..Hos
0080 74 3a 20 77 77 77 2e 6e 6f 6d 62 72 65 6f 72 67  t: www.n ombreorg
0090 61 6e 69 7a 61 63 69 6f 6e 39 34 34 38 2e 6f 72  anizacio n9448.or
00a0 67 0d 0a 55 73 65 72 2d 41 67 65 6e 74 3a 20 4d  g..User- Agent: M
00b0 6f 7a 69 6c 6c 61 2f 35 2e 30 20 28 58 31 31 3b  ozilla/5 .0 (X11;
00c0 20 55 62 75 6e 74 75 3b 20 4c 69 6e 75 78 20 78  Ubuntu; Linux x
00d0 38 36 5f 36 34 3b 20 72 76 3a 34 37 2e 30 29 20  86.64; r v:47.0)
00e0 47 65 63 6b 6f 2f 32 30 31 30 30 31 30 31 20 46  Gecko/20 100101 F
00f0 69 72 65 66 6f 78 2f 34 37 2e 30 0d 0a 41 63 63  irefox/4 7.0..Acc
0100 65 70 74 3a 20 2a 2f 2a 0d 0a 41 63 63 65 70 74  ept: /*/ ..Accept

```

Figura 23: Intercambio HTTP con peticiones GET y respuestas del servidor.

7.2. Intercambio DNS + acceso HTTPS

El cliente resuelve el nombre del servidor antes de iniciar la conexión segura. A continuación se muestra la DNS Query y su respuesta con filtro `dns || ssl`:

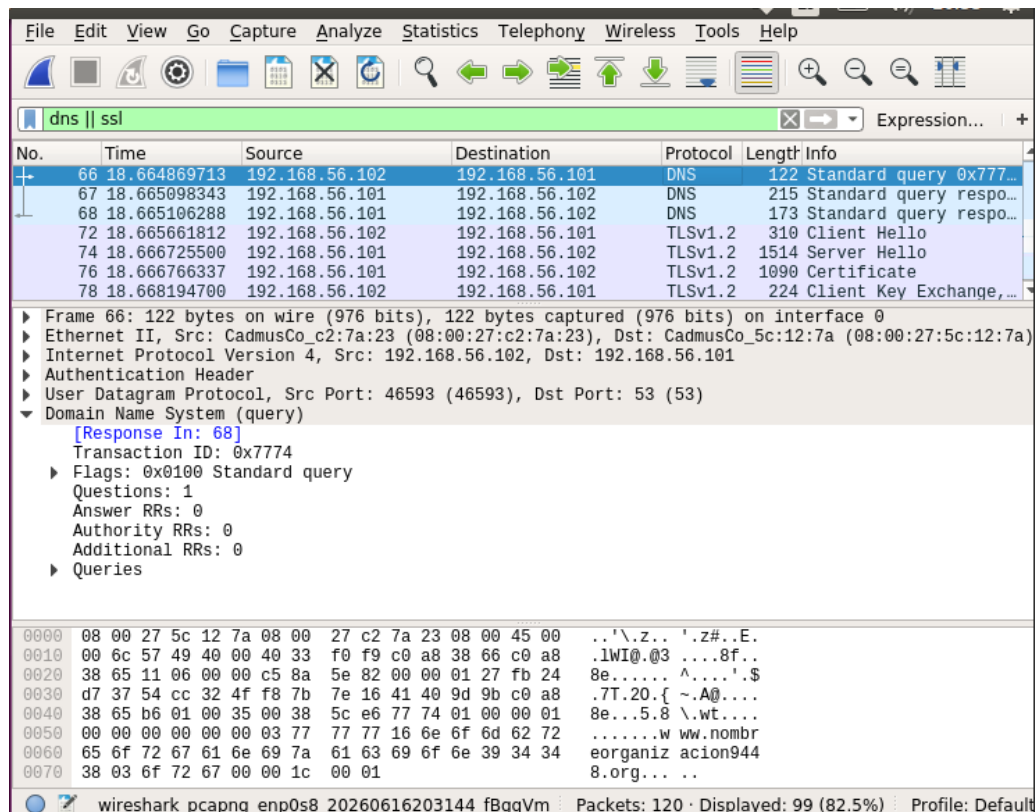


Figura 24: DNS Query previa al acceso HTTPS.

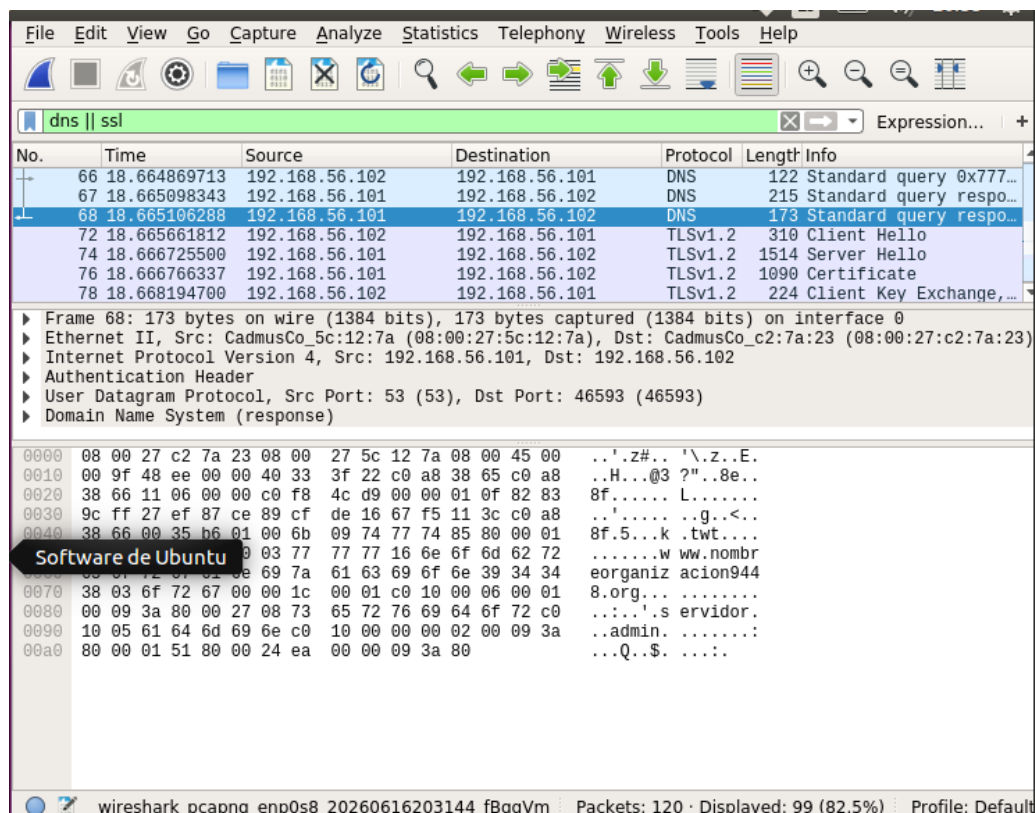


Figura 25: DNS Response con la dirección IP resuelta.

Tras la resolución, el cliente inicia el handshake TLSv1.2 con el mensaje **Client Hello**, indicando las cipher suites soportadas:

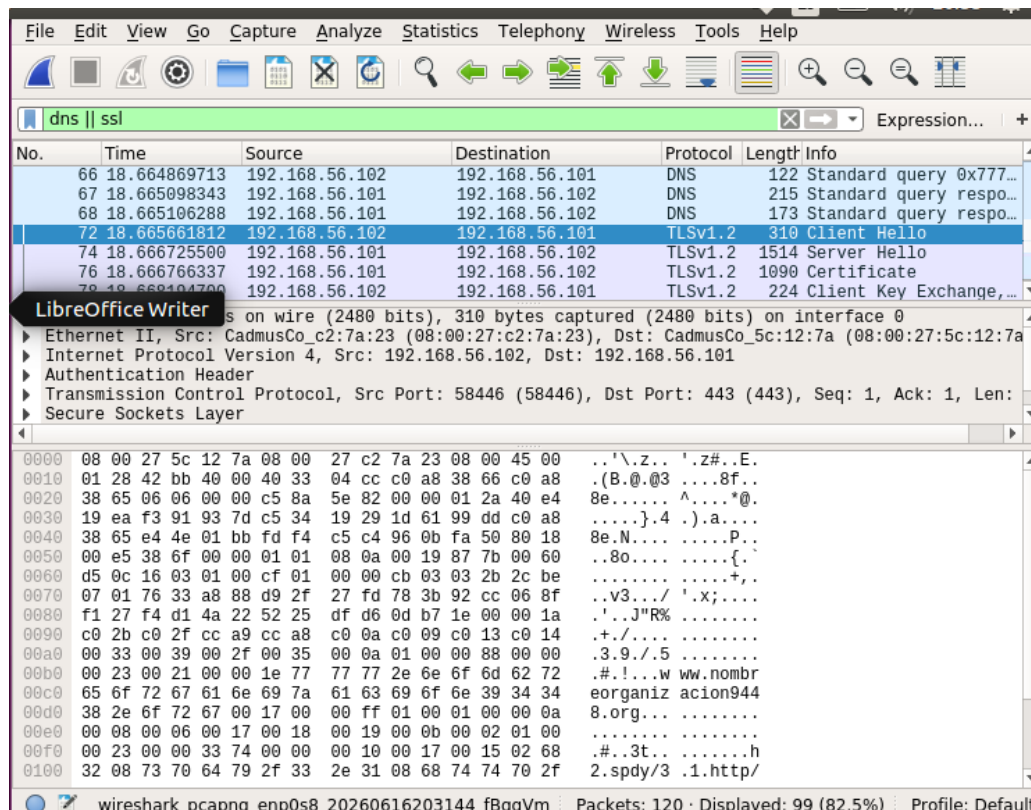


Figura 26: Mensaje Client Hello del handshake TLSv1.2.

El servidor responde con **Server Hello**, seleccionando la cipher suite y adjuntando su certificado X.509:

No.	Time	Source	Destination	Protocol	Length	Info
66	18.664869713	192.168.56.102	192.168.56.101	DNS	122	Standard query 0x777...
67	18.665098343	192.168.56.101	192.168.56.102	DNS	215	Standard query respo...
68	18.665106288	192.168.56.101	192.168.56.102	DNS	173	Standard query respo...
72	18.665661812	192.168.56.102	192.168.56.101	TLSv1.2	310	Client Hello
74	18.666725500	192.168.56.101	192.168.56.102	TLSv1.2	1514	Server Hello
76	18.666766337	192.168.56.101	192.168.56.102	TLSv1.2	1090	Certificate
78	18.668194700	192.168.56.102	192.168.56.101	TLSv1.2	224	Client Key Exchange...

Frame 74: 1514 bytes on wire (12112 bits), 1514 bytes captured (12112 bits) on interface 0
 Ethernet II, Src: CadmusCo_5c:12:7a (08:00:27:5c:12:7a), Dst: CadmusCo_c2:7a:23 (08:00:27:c2:7a:23)
 Internet Protocol Version 4, Src: 192.168.56.101, Dst: 192.168.56.102
 Authentication Header
 Transmission Control Protocol, Src Port: 443 (443), Dst Port: 58446 (58446), Seq: 1, Ack: 213, Len: 1514
 Secure Sockets Layer

0000 08 00 27 c2 7a 23 08 00 27 5c 12 7a 08 00 45 00 ..'.z#.. '\.z..E.
 0010 05 dc 30 53 40 00 40 33 12 80 c0 a8 38 65 c0 a8 ..0S@.038e..
 0020 38 66 06 06 00 00 c0 f8 4c d9 00 00 01 12 db 2d 8f..... L.....-
 0030 54 ee 1c d6 d7 55 43 7c 72 3b 21 9c 48 52 c0 a8 T....UC| r;!..HR..
 0040 38 66 01 bb e4 4e 96 0b fa 50 fd f4 c6 98 80 10 8f...N.. .P.....
 0050 00 eb a0 01 00 00 01 01 08 0a 00 60 d5 0c 00 19
 0060 87 7b 16 03 03 00 50 02 00 00 4c 03 03 8a 58 67 ..{...P. ..L...Xg
 0070 ed 3b c6 25 3f f2 a4 0f b8 98 98 c7 e9 6f 22 e3 .;.%?... ..o".
 0080 99 24 f7 ba a8 a4 b2 63 e2 d5 cb e9 c0 00 c0 2f .\$.....c/
 0090 00 00 24 00 00 00 00 ff 01 00 01 00 00 0b 00 04 ..\$.
 00a0 03 00 01 02 00 23 00 00 00 10 00 0b 00 09 08 68#.....h
 00b0 74 74 70 2f 31 2e 31 16 03 03 07 b3 0b 00 07 af ttp/1.1.
 00c0 00 07 ac 00 03 e5 30 82 03 e1 30 82 02 c9 a0 030. ..0.....
 00d0 02 01 02 02 01 01 30 0d 06 09 2a 86 48 86 f7 0d0. ...*.H...
 00e0 01 01 0b 05 00 30 75 31 0b 30 09 06 03 55 04 060u1 .0...U..
 00f0 13 02 45 53 31 0f 30 0d 06 03 55 04 08 0c 06 4d ..ES1.0. ..U....M
 0100 75 72 63 69 61 31 0c 30 0a 06 03 55 04 0a 0c 03 urcia1.0 ...U....

Figura 27: Mensaje Server Hello con el certificado del servidor.

7.3. Intercambios DNS, SMTP y POP

Antes del envío del correo, el cliente consulta el registro MX del dominio para localizar el servidor de correo. La siguiente captura muestra dicha resolución DNS junto con el inicio del intercambio SMTP:

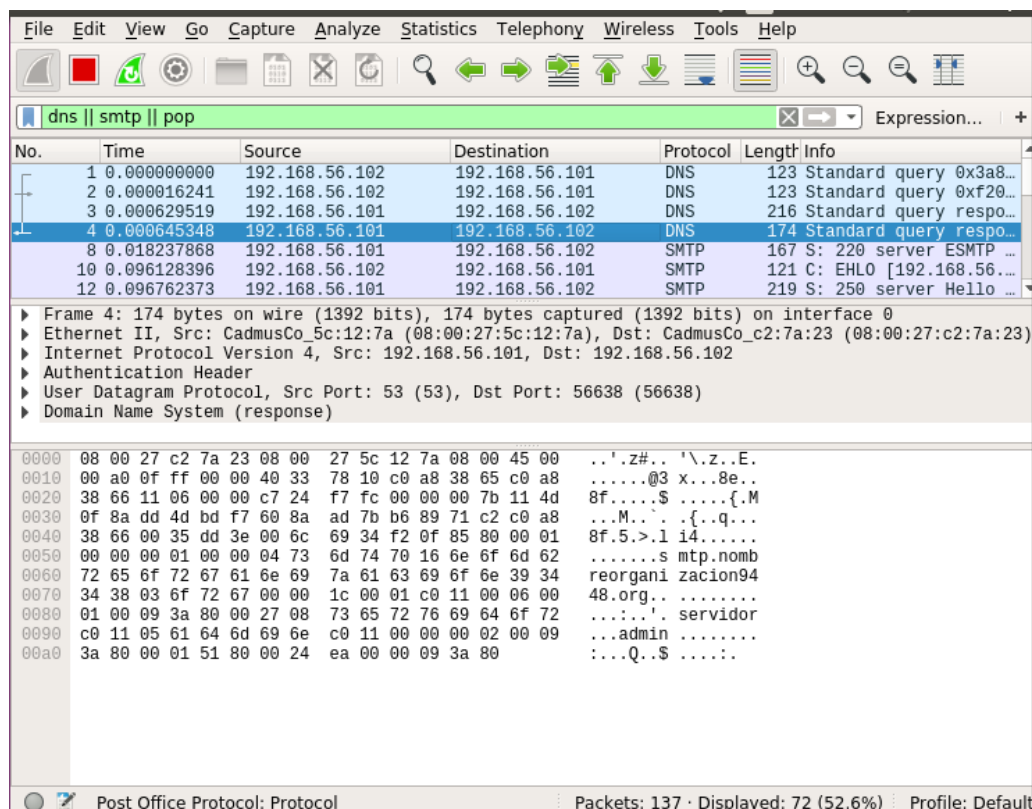


Figura 28: DNS Query del registro MX e inicio del intercambio SMTP.

A continuación se muestra el intercambio SMTP completo, donde se puede observar los comandos EHLO, MAIL FROM y RCPT TO entre cliente y servidor:

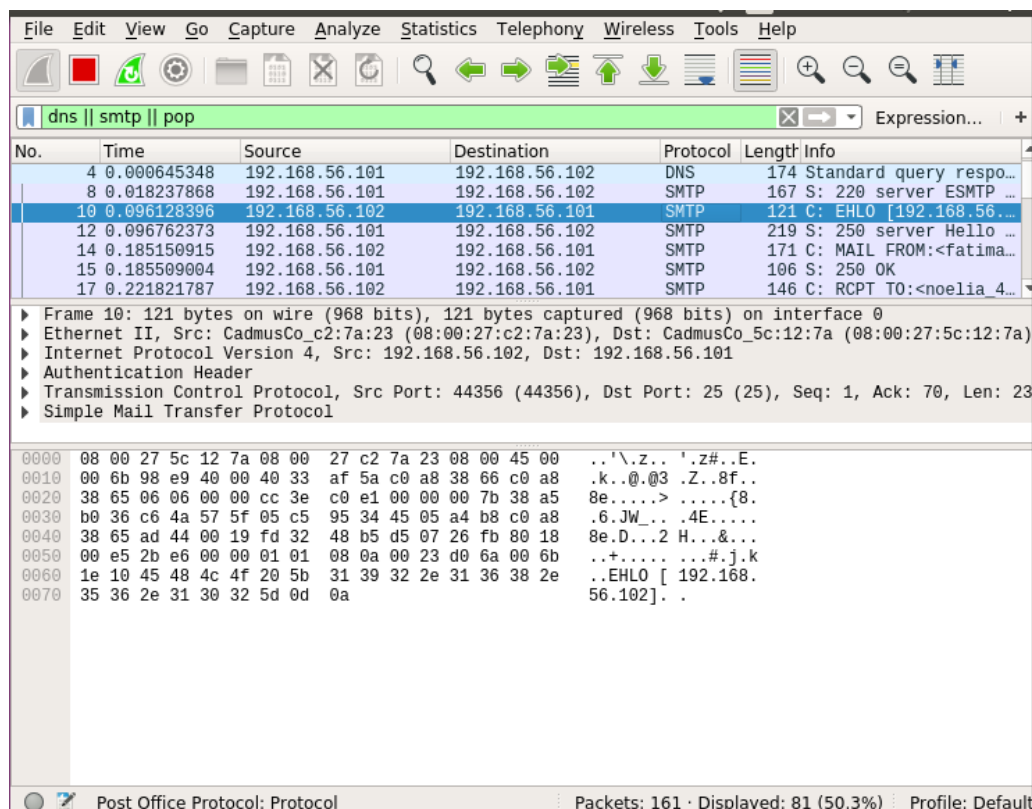


Figura 29: Intercambio SMTP durante el envío de correo.

Para la recepción del correo, el cliente se conecta al servidor mediante POP. Se puede observar el comando USER noelia_48 seguido del proceso de autenticación:

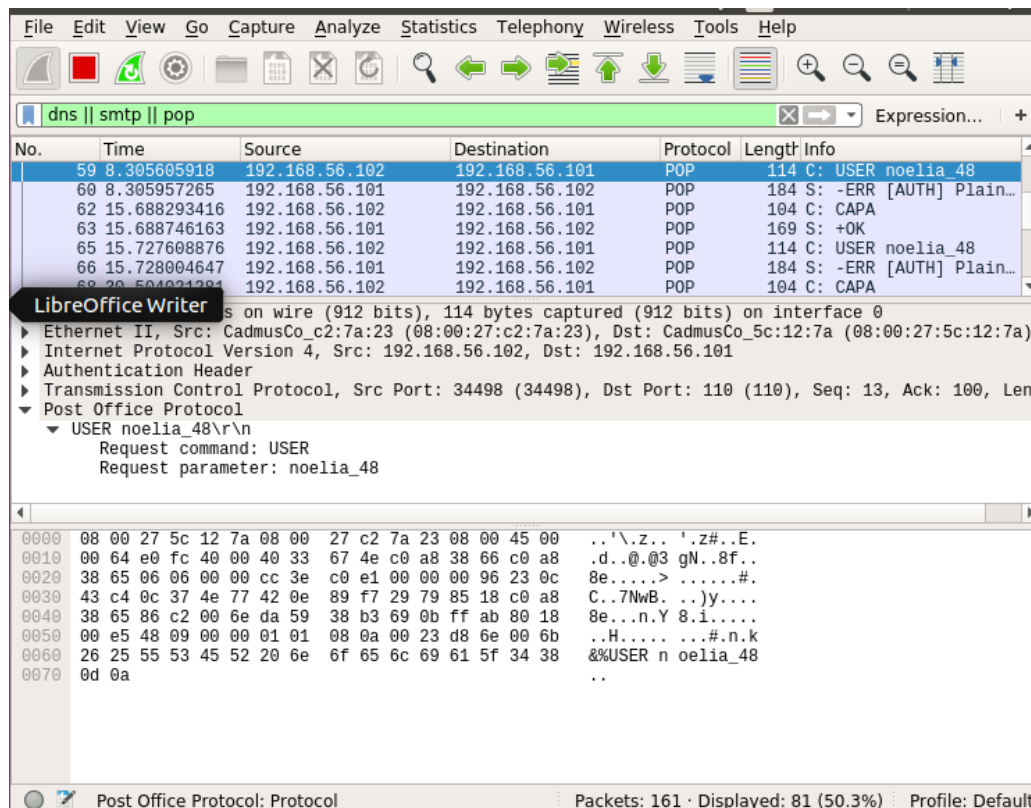


Figura 30: Intercambio POP durante la recepción de correo por noelia_48.

8. Problemas encontrados

El principal problema surgió en el servidor HTTP Python. Al ser el primer servicio implementado en la práctica, el desconocimiento del funcionamiento interno de un servidor web dificultó el arranque. Conceptos como la gestión manual de cabeceras, las cookies o la persistencia no quedaban del todo claros hasta que se ponían en práctica.

9. Conclusión y valoración personal

La práctica permite consolidar de forma práctica los contenidos teóricos de la asignatura, ya que ambos avanzan en paralelo a lo largo del curso. Implementar servicios como DNS, correo o IPSec en un entorno real, aunque reducido, facilita la comprensión de su funcionamiento de una manera que la teoría por sí sola no consigue.

A nivel personal, la experiencia ha sido positiva. La dificultad es progresiva y los recursos del aula virtual resultan suficientes para resolver las dudas que van surgiendo a lo largo del desarrollo.

Horas de trabajo

- Sesiones de laboratorio: 20 horas (12 sesiones de 1 hora y 40 minutos).
- Trabajo autónomo: 14 horas aproximadamente.
- Verificaciones: 3 horas (1 hora por verificación).
- **Total: 37 horas aproximadamente.**